

Introduction to the Microsoft Identity Platform and Entra ID

Rob Windsor
rob@robwindsor.com
@robwindsor

About Me



Rob Windsor

.NET/Microsoft 365 developer, trainer, author
Microsoft MVP | Microsoft 365 Development



Twitter: <https://twitter.com/robwindsor>

LinkedIn: <https://www.linkedin.com/in/rwindsor>

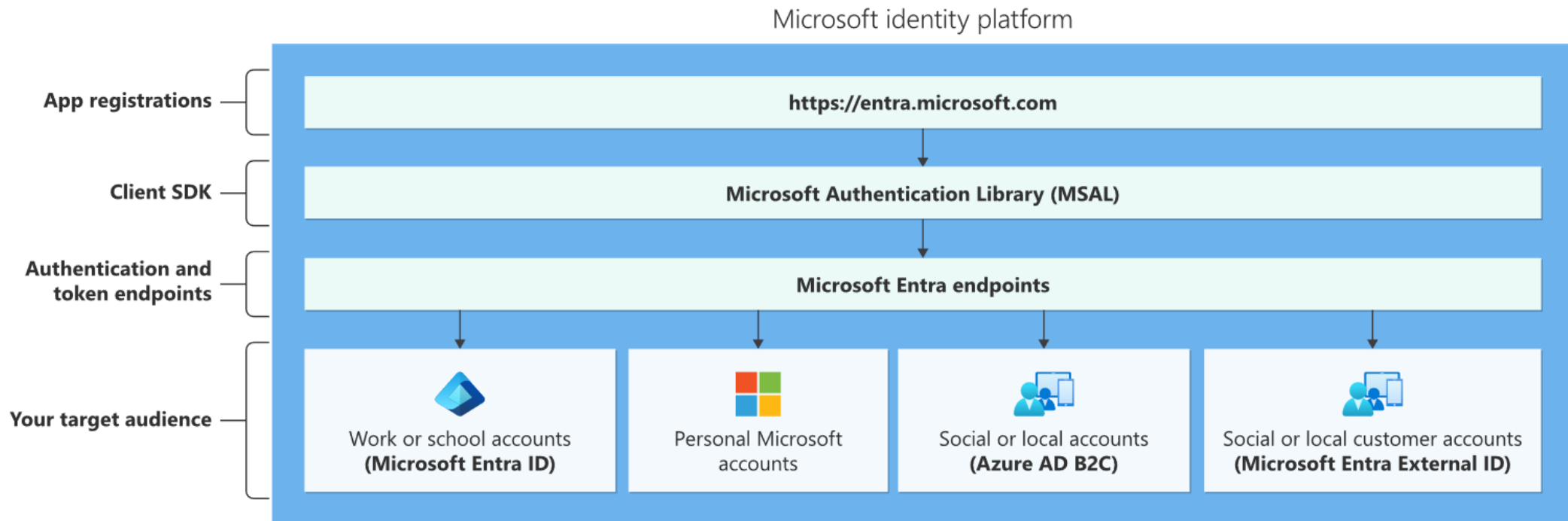
Blog: <https://robwindsor.hashnode.dev>

YouTube: <https://www.youtube.com/@RobWindsor>

GitHub: <https://github.com/rob-windsor>

Microsoft Identity Platform

The Microsoft identity platform enables developers to authenticate identities and obtain tokens from Microsoft Entra ID. Applications use it to sign users in and obtain authorization to call protected APIs.



Microsoft Identity Platform

For applications it provides:

- Application registration and configuration
- Authentication of users and workloads
- Token issuance
- Consent and permission management
- Authorization to access Microsoft and custom protected APIs
- Authentication libraries and SDKs such as MSAL

Built on open standards

- OAuth 2.0 and OpenID Connect

Key Concepts

Tenant

Organization's Entra ID directory containing identities, applications, and access configuration

User

A person whose identity is represented in the tenant

Application

Software that needs to authenticate users or access protected resources

Application Registration

Entra ID configuration that represents a software application to the Microsoft identity platform

Service Principal

The Entra ID application's identity within a specific tenant

Entra ID Authentication and Authorization

Entra ID is the identity provider behind the Microsoft Identity Platform

Authentication verifies the identity of the caller

Authorization determines what the caller is able to access or do

Microsoft Entra ID supports both authentication and authorization

Registering an Application

Software applications that use the Microsoft identity platform are represented by an application (app) registration in Microsoft Entra ID

The app registration defines how the identity platform identifies and interacts with the application

Supported account types:

- Single tenant: only accounts in your organization
- Multi-tenant: accounts in any organization
- Personal Microsoft accounts: consumer accounts (e.g. Outlook.com)

Redirect URIs

- Where the identity platform is allowed to return an authentication or authorization response

App Registration Configuration

Identifiers

- Tenant ID: identifies the Entra tenant
- Client (Application) ID: identifies the app registration

Authentication configuration

- Client Secret or Certificate: credential a confidential application can use to prove its identity
- Redirect URI: location(s) where Entra ID is allowed to return an authentication or authorization response

Authorization Configuration

- Permission Requests: requested permissions to resources or APIs
 - Granted through user or administrator consent

App Registration Manifest

JSON representation of an app registration

Contains the app registration's configuration and attributes

Can be viewed and edited in App registrations → Manifest

Some settings are easier—or sometimes only possible—to configure through the manifest

The screenshot shows the Azure AD portal interface for editing an app registration manifest. The breadcrumb path is "Home > Rob Windsor's Developer Tenant | App registrations > Test Client". The page title is "Test Client | Manifest". A search bar and navigation links (Refresh, Got feedback?) are at the top. A left-hand navigation pane lists various app configuration options, with "Manifest" selected. A blue informational banner at the top of the main content area states: "Starting from as early as 9/1/2026, and no later than 10/1/2026, you will no longer be able to view, save, upload, or download the Azure AD Graph app manifest on this page. Please follow the steps described in [manifest migration guide](#) to ensure a smooth migration." Below this, a paragraph explains that an app manifest is a JSON representation of an app registration and that both Microsoft Graph and AAD Graph manifests represent the same settings in different formats. Two tabs are visible: "Microsoft Graph App Manifest (New)" and "AAD Graph App Manifest (Deprecating Soon)". A link points to "Understanding the Microsoft Graph application manifest". Action buttons for "Save", "Discard", "Download", and "Upload" are present. The main area displays a JSON manifest for the "Test Client" app, with line numbers 1 through 13 on the left. The JSON content is:

```
1 {
2   "id": "94c86109-25c9-4142-84b7-2d7a1c10459e",
3   "deletedDateTime": null,
4   "appId": "81086d61-cdca-45d4-a835-0960f7b71bc7",
5   "applicationTemplateId": null,
6   "disabledByMicrosoftStatus": null,
7   "createdByAppId": "18ed3507-a475-4ccb-b669-d66bc9f2a36e",
8   "createdDateTime": "2026-10-02T14:19:59Z",
9   "displayName": "Test Client",
10  "description": null,
11  "groupMembershipClaims": null,
12  "identifierUri": [],
13  "isDeviceOnlyAuthSupported": null,
```

Delegated and Application Permissions

Delegated Permissions

- A signed-in user is present
- The application acts on behalf of that user
- Effective access is limited by both:
 - what the user is allowed to do
 - what delegated permissions the Entra ID application has been granted
- Delegated scopes appear in the scp claim of the access token

Application Permissions

- Also known as app-only permissions
- No signed-in user is required
- The application acts as itself
- Access is based on the application permissions granted to the Entra ID application
- Application permissions require administrator consent
- In app-only access tokens, application permissions appear in the roles claim

Permission Consent

User consent

- A user grants eligible delegated permissions to an application
- Applies to that user's use of the application

Administrator consent

- An administrator grants permissions on behalf of the organization
- Required for application permissions
- Some delegated permissions also require administrator consent

Consent creates a permission grant that allows the application to receive access tokens for the approved permissions

Configuring Static Permission Requests

Home > Rob Windsor's Developer Tenant | App registrations > Test Client

Test Client | API permissions

Search

Refresh

Got feedback?

Overview

Quickstart

Integration assistant

Diagnose and solve problems

Manage

Branding & properties

Authentication (Preview)

Certificates & secrets

Token configuration

API permissions

Expose an API

App roles

Owners

Roles and administrators

Manifest

Support + Troubleshooting

Granting tenant-wide consent may revoke permissions affected. [Learn more](#)

The "Admin consent required" column shows the organization, or in organizations where this app is used

Configured permissions

Applications are authorized to call APIs when they are granted all the permissions the application needs. [Learn more](#)

+ Add a permission

✓ Grant admin consent for this app

API / Permissions name	Type
Microsoft Graph (1)	
User.Read	Delegated

To view and manage consented permissions for individual users, go to the [Users](#) blade.

Request API permissions

< All APIs

Microsoft Graph
<https://graph.microsoft.com/> [Docs](#)

What type of permissions does your application require?

Delegated permissions

Your application needs to access the API as the signed-in user.

Application permissions

Your application runs as a background service or daemon without a signed-in user.

Select permissions [expand all](#)

Files.Read

Permission	Admin consent required
Files (1)	
☒ Files.Read.All ⓘ Read files in all site collections	Yes
☐ Files.ReadWrite.All ⓘ	Yes

Add permissions

Discard

Dynamic Permission Requests

Delegated permissions requested at runtime

Permissions requested at runtime using a scope parameter

Only applies to delegated permissions

Supports incremental consent

- Additional permissions can be requested as needed

Tokens

ID tokens

- Confirms user authentication
- Contains identity claims
- Should not be used for authorization

Access tokens

- Authorizes access to an API
- Sent as a bearer token in the Authorization header of a request
- Generally short lived

Refresh tokens

- Longer-lived token used to acquire new access tokens silently

ID tokens are JWTs. Access tokens are often JWTs, but client apps should treat them as opaque

Sites like jwt.ms can be used to inspect JWT token contents during development and debugging

Access Token Versions

Entra ID supports two versions of access tokens

v1.0

- Legacy/default format for many Entra-only APIs
- ver claim is 1.0
- Uses v1 metadata for validation

v2.0

- Newer token format
- ver claim is 2.0
- Uses v2 metadata for validation

The requestedAccessTokenVersion property of the resource API's app registration configuration determines the access token version. The default value of this property is v1.0.

Validating Access Tokens

APIs should validate incoming access tokens

Validate the access token:

- Signature: issued by a trusted authority
- Audience (aud): must identify your API
 - v1.0: may be an Application ID URI/resource identifier
 - v2.0: the API's client ID
- Issuer (iss): must match the expected Entra ID issuer/tenant
- Lifetime (exp): token must not be expired

Authorize the caller:

- Scope (scp): required delegated scope
- Roles (roles): required application permission or app role

Use supported middleware or libraries to validate signature, issuer, audience, and lifetime. Then enforce the required scopes or roles for authorization. Microsoft.Identity.Web provides this integration for ASP.NET Core APIs.

Authentication Libraries

Microsoft Authentication Library (MSAL)

- Acquires tokens from the Microsoft identity platform
- Supports user and application authentication scenarios
- Direct token-acquisition API

Azure Identity

- Provides TokenCredential implementations
- Designed to integrate naturally with Azure SDK clients
- Supports developer credentials, workload identity, managed identity, etc.

Microsoft Identity Web

- Integrates Microsoft identity with ASP.NET Core web apps and APIs
- Wraps MSAL.NET and integrates with ASP.NET Core middleware and dependency injection
- Handles token validation in APIs automatically via `AddMicrosoftIdentityWebApi()`

Microsoft Authentication Library (MSAL)

Recommended library for acquiring tokens from the Microsoft Identity Platform

Available for: .NET, JavaScript/TypeScript, Python, Java, iOS, Android, Go (Preview)

Two application types:

- Public client: Cannot securely hold an application credential
- Confidential client: Can securely hold an application credential

Built-in token cache with silent acquisition

- Checks the cache before acquiring a token
- If a valid access token exists, it is returned immediately
- Refreshes tokens silently when possible

NuGet: Microsoft.Identity.Client

Azure Identity Library

Higher-level credential abstraction used with Azure SDK clients

Built around the TokenCredential model

Supports multiple Entra ID authentication mechanisms

Local development

- Uses DefaultAzureCredential
- Can use credentials from Visual Studio, Azure CLI, etc.

Azure-hosted

- Prefer a deterministic credential such as ManagedIdentityCredential
- Managed identity avoids storing application credentials

NuGet: Azure.Identity

Authentication Flows

The Microsoft Identity Platform supports several OAuth 2.0 / OpenID Connect flows

Choosing the right flow depends on your app type and use case

Flow	App Types	User	Tokens
Authorization Code	Web apps, SPAs, mobile	Yes	ID, access, and refresh
Client Credentials	Daemons, services	No	Access
Device Code	CLIs, IoT, browserless	Yes	ID, access, and refresh
On-behalf Of	Middle tier	Upstream	Access

Authorization Code Flow

Used in user-facing applications like web apps, SPAs, mobile apps, and desktop apps

Can only use delegated permissions

Flow:

- Application sends the user to the Entra ID authorization endpoint
- User authenticates and consents
- Entra ID returns an authorization code to the registered redirect URI
- Application sends the authorization code to the token endpoint
- Token endpoint returns an access token and, when requested, a refresh token

Confidential clients authenticate themselves during token redemption; public clients do not use a client secret

Authorization Code Flow with PKCE (Proof Key for Code Exchange)

Adds proof of possession to the Authorization Code flow to protect against authorization-code interception

- Application generates a random code verifier
- Application derives a code challenge
- Authorization request includes the challenge
- Token request includes the verifier
- Entra ID checks that the verifier matches the original challenge

Required for SPAs and recommended for native and confidential clients

Client Credentials Flow

Used in non-interactive applications: background services, daemons, API-to-API calls

No signed-in user: application acts as itself

Can only use application permissions

Flow:

- Application sends a request to the Entra ID token endpoint
- Application authenticates using an application credential
 - Secret, certificate, or federated credential
- Request uses {resource}/.default scope
- Entra ID returns an access token
 - No refresh token issued

On-behalf-of (OBO) Flow

Used when a web API needs to call another API on behalf of the signed-in user

Flow:

- Client application obtains an access token for the middle-tier API
- Client calls the middle-tier API with that access token
- Middle-tier API exchanges the incoming token for a new access token for the downstream API
- Middle-tier API calls the downstream API using the new token

Key points:

- Uses delegated permissions
- Preserves the user's identity and delegated permissions through the call chain
- The downstream token is issued for the downstream API, not reused from the original request
- OBO does not elevate the user's permissions

Exposing an API

To protect your own API with Entra ID, configure its app registration to expose the permissions that client applications can request. This is part of an application registration configuration.

- Application ID URI: uniquely identifies the API as a resource
 - Set in the "Expose an API" section of your app registration
- Scopes: define delegated permissions exposed by the API (e.g. Forecast.Read)
 - Appear as scp claims in access tokens
- App roles: define application permissions your API exposes for app-only access
 - Appear as roles claims in access tokens
- Authorized client applications: optionally pre-authorize trusted client applications for selected delegated scopes

Client applications request your API's permissions the same way they request permissions for Microsoft APIs

Resources

Microsoft identity platform documentation

- learn.microsoft.com/en-us/entra/identity-platform/

Implement user authentication and authorization

- learn.microsoft.com/en-us/training/paths/az-204-implement-authentication-authorization/

Microsoft identity platform code samples

- learn.microsoft.com/en-us/entra/identity-platform/sample-v2-code

Calling Downstream APIs on Behalf of Your Users with Entra ID and the OBO Flow

- robwindor.hashnode.dev/calling-downstream-apis-on-behalf-of-your-users-with-entra-id-and-the-obo-flow